

Rocket Mission Simulator - Implementation Plan

Priority key: P0 = must-have for MVP / course goals | P1 = important | P2 = nice to have | P3 = stretch

MVP target: A Swing application where the user starts a mission, sees a rocket launch and follow a simplified automatic flight, watches live telemetry, responds to mission events, reaches the target orbit, deploys the satellite, and receives a success/failure outcome.

Implementation order

Order	Task	Priority	What to do
1	Create the basic GUI skeleton	P0	Create the main Swing window and the main areas of the interface: simulation area, telemetry area, event/mission area, and mission log area. These can initially contain placeholder values.
2	Display a static rocket and basic environment	P0	Draw a simple 2D rocket in the simulation area, with a basic background. No animation or polish is needed yet.
3	Create the mission states using enums	P0	Define the main mission states, for example PRE_LAUNCH, ASCENT, ORBIT, DEPLOYMENT, SUCCESS and FAILURE. Store the current mission state.
4	Connect the Launch button to the mission state	P0	Pressing Launch changes the mission from PRE_LAUNCH to ASCENT and gives visible feedback in the GUI.
5	Implement simple automatic rocket flight	P0	During ASCENT, update basic values such as altitude and velocity and move the rocket upward. Manual throttle and tilt are not required yet.
6	Display live telemetry and integrate multithreading	P0	Show changing values such as altitude and velocity in the GUI. Integrate the approved multithreading/concurrency topic here so telemetry updates correctly while the GUI remains responsive.
7	Detect the target orbit	P0	Use simplified conditions, such as altitude and velocity thresholds, to determine when the target orbit has been reached and change the mission state.
8	Deploy the satellite	P0	After the orbit state is reached, allow or automatically trigger satellite deployment and update the mission state.
9	Display mission success and failure	P0	End the mission with a clear success or failure result and stop normal mission progression.
10	Implement the mission-event system	P0	Allow events to occur during the mission, display them in the GUI, and let the user choose a response.
11	Implement event recovery and consequences	P0	User responses can recover from an event, affect mission state, or cause failure.
12	Add mission log	P0	Record important events such as launch, warnings, event responses, orbit achievement, deployment, and outcome.
13	Add Restart	P1	Reset the mission state, rocket values, telemetry, events, mission log, and GUI so a new mission can begin.

Order	Task	Priority	What to do
14	Add atmosphere-to-space visual changes	P1	Change the background as altitude increases so the flight visually transitions from atmosphere to space.
15	Add throttle slider	P2	Allow the player to change throttle manually.
16	Make throttle affect thrust and fuel	P2	Throttle changes the rocket's simulated acceleration and fuel consumption.
17	Make fuel consumption affect mission success	P2	Track the player's fuel use. If too much fuel is used and the rocket runs out before completing the mission, transition the mission to FAILURE.
18	Add left/right tilt control	P3	Allow the player to change rocket orientation.
19	Make tilt affect trajectory	P3	Rocket orientation changes the direction of travel so the player can steer toward the mission target.
20	Require the rocket to reach a target point in orbit	P3	Extend the basic orbit objective so success requires reaching a specific point or region of the target orbit. The player uses tilt to guide the rocket toward it.
21	Make excessive tilt cause mission failure	P3	Let tilt influence mission state. If the rocket exceeds a chosen safe tilt/orientation limit, transition the mission to FAILURE.
22	Add camera following and zoom	P3	Move or scale the view so the rocket stays visible during flight.
23	Final testing, cleanup and documentation	P0	Test the complete mission, fix bugs, clean up code, check cross-platform execution, and prepare the README and final submission files.